How to build multiple AI agents work together

Akash
@Akasheth_ · 20h
Follow

everyone is talking about AI agents but most people are still using a single agent.

the biggest shift happening in AI right now isn't making one agent smarter, it's getting multiple agents to work together.

think of it like hiring employees. one brilliant person handling research, writing, analysis, review and coding all at once but hire five specialists, each focused on one thing and you get excellent results across the board.

multi-agent AI works the same way. Instead of one model doing everything sequentially, you have a team of specialized agents running simultaneously, each doing exactly one job, passing their output to the next.

netflix uses this to analyze hundreds of build logs simultaneously. Harvey (the legal AI company) coordinates complex legal work across many documents at once. shopify is

pushing toward 90% autonomous coding by Q3 2026 with this exact approach.

Before you build

The three patterns every multi-agent system uses

all multi-agent setups follow one of three structures. Pick the right pattern before you write a single line of code.

Use the pipeline when tasks must happen in order. Use the fan-out when one big job can be split into parallel chunks. Use the specialist team when different domains need to collaborate on one deliverable. Most real systems combine two or more of these.

how to build your agent team

1.Plan your team on paper, before any code

Ask yourself four questions. What is the overall goal? What are the distinct sub-tasks? Which ones can run at the same time? What specialist would you hire for each? Every specialist you would hire becomes one agent.

2. Design each agent: role, tools, output format

Every agent needs exactly three things defined. Get any one of them wrong and the whole system breaks down.

First, a clear and narrow role written as a system prompt. The narrower, the better. Second, specific tools only give each agent only what it needs. A pricing analyst needs web access; a report writer needs file access. Third, a defined output format. This is the most important decision you will make. It is how agents communicate with each other.

> Rule: standardize output formats before building anything. If Agent A outputs free text and Agent B expects JSON, the handoff silently fails and every downstream agent produces garbage.

3. Build the orchestration (who runs when)

The orchestration layer is your traffic controller. It decides which agents run in parallel, which wait, how data passes between them, and what happens when something fails. Always define failure behavior explicitly for example: "If the pricing agent cannot access a competitor's website, log the failure and continue with historical data. Do not halt the whole pipeline."

4. Enable dreaming agents that remember and improve

Dreaming is a background process that runs between sessions. It reviews past work, extracts patterns, and updates the agent's memory automatically, without you touching any prompts. Your team gets smarter over time on its own.

Harvey reported a 6x increase in task completion rates just from enabling this, not from a new model, purely from agents learning across sessions. Run it nightly so your team has a full day of sessions to learn from before updating memory.

5. Define outcomes, tell agents what "good" looks like

Instead of hoping agents produce good output, write a rubric. The agent evaluates its own output against your criteria and iterates until it passes. Errors get caught before you ever see the result. If it cannot pass after several attempts, it flags the failure with a reason rather than silently delivering substandard work

6. Start with two agents, not ten

This is where most people go wrong. They design the full 10-agent system on day one, run it, hit four simultaneous failures, and cannot tell which agent caused which problem.

A two-agent system that actually works is more valuable than a ten-agent system that fails in ways you cannot diagnose. The temptation is to go big immediately. Resist it.

7. Monitor, tune, and grow

once your team is running reliably, treat it like a real team. Watch how it works. Notice where it slows down. Fix the bottlenecks. Ask yourself: which agents fail most often and why? Which handoffs produce the most downstream errors? Are the output formats still serving their purpose? Have the role definitions stayed narrow, or has scope crept in?

A well-maintained agent team compounds over time. Each improvement makes the whole system better. The agents that learn via dreaming bring those improvements into every future session automatically.

a few things worth knowing before you scale: token costs multiply with agents, so build a rough cost model before adding more. Latency stacks in sequential pipelines, if speed matters, favor parallel patterns. Human review still matters, so build

checkpoints where a person can inspect output before the pipeline continues, especially for high-stakes work.

the teams getting the most out of multi-agent systems are not the ones who built the biggest teams fastest. they are the ones who started small, understood every part of what they built, and grew deliberately.

start with two agents, ship something then make it better.

Akash
@Akasheth_
developer // content creator
Follow